Nafs-Aql

A verifier-gated dual-process architecture on small Qwen models, CPU-only

Dirac R&D Team

18 October 2026, v2.0

Outline

1. Motivation and constraints
2. Concept: the classical self as design metaphor
3. Formal model and results
4. Architecture
5. Implementation stack and measured feasibility
6. Specification highlights
7. Roadmap and gates
8. **The system as built and measured results**
9. Evaluation plan and the road ahead

Status (18 October 2026)

Phases 0–6 are implemented on the CPU reference machine and Phase 7 (CPU inference performance) was added; GPU work is deferred. Results are *measured* on a shared four-core host with small samples, so they are indicative.

Motivation

Why a dual-process system

Observation

Reasoning models get hard questions right by spending inference compute (long traces, many samples, verification) and waste it on easy ones. Competence is knowing when to switch.

Constraints of this project

- Pretrained Qwen models, small
- CPU-only machine (4 cores, AVX2, 32 GB)
- Unsloth for fine-tuning
- Rust web app: chat, performance tests, evaluation

What the draft had

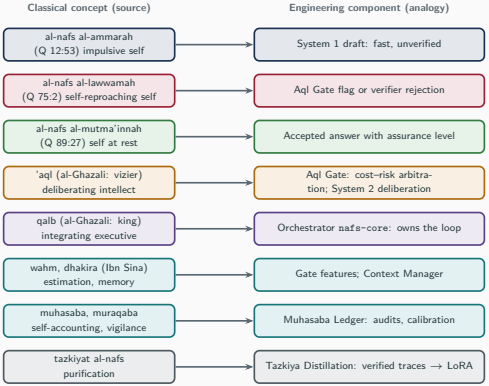
Twelve sound validation results, a 27B/35B GPU roster, no feasibility study, no specification, no roadmap.

What this review delivers

Faithful concept grounding; 17 formal results with engineering consequences; verified stack; measured CPU numbers; spec with 94 identified requirements; 7-phase roadmap with gates.

Concept

The classical self as a design metaphor

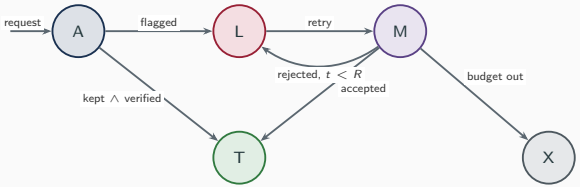


- Qur'anic stages used as mnemonics: commanding (12:53), self-reproaching (75:2), at rest (89:27)
- al-Ghazali: heart as king, intellect as vizier, appetite and anger as soldiers
- Ibn Sina: estimation (*wahm*) and memory as internal senses
- *muhasaba*: self-correction *under a standard*

Analogy only

No claim about the soul, consciousness or authority. The *ruh*, divine inspiration and the Active Intellect are not mapped. Scholar review is a release gate.

One request as a bounded automaton



A draft (fast path); L flagged by gate or verifier; M deliberate retry conditioned on verifier evidence; T accepted (assurance level attached); X abstain with uncertainty notice.

- A: fast draft, unverified
- L: flagged by the gate or a verifier
- M: deliberate retry on verifier evidence
- T: accepted, with an assurance level
- X: abstain with an uncertainty notice

Lemma 1 (termination)

At most $1 + N(R+1)$ model calls per request: a hard latency bound, enforced by the gateway.

Formal model

Objects and objective

Task x, answer y, correctness $\kappa(x, y)$. Fast path π_1 (cost c_1, accuracy p_1); slow path $\pi_2(\cdot \mid x, b)$ with thinking budget b; verifier with true-accept r and false-accept φ; gate $\gamma(s)$ on features s; loss L per wrong answer, L_a per abstention, price λ per cost unit.

$$\min_{\gamma, b, N, R} \; \mathbb{E}_x \big[L\, \mathbb{P}[\text{wrong}] + L_a\, \mathbb{P}[\text{abstain}] + \lambda\, \mathbb{E}[\text{cost}] \big]$$

Labels

[exact] no further hypotheses; **[strong]** needs an empirical check; **[conjecture]** scheduled hypothesis.

Rule

Every result ends with an engineering consequence: a parameter, an algorithm or a gate in the specification.

Theorem 1: precision is bounded by the verifier

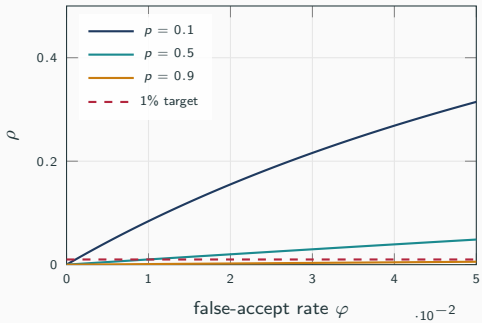
Theorem (Verified-return precision, exact)

$$\rho(p) = \mathbb{P}[\textit{wrong} \mid \textit{accepted}] = \frac{\varphi(1-p)}{\varphi(1-p) + rp},$$
$$\textit{and } \rho \le \varepsilon \iff \varphi \le \frac{\varepsilon r p}{(1-\varepsilon)(1-p)}.$$

Hard tasks ($p = 0.1$), 1% target: $\varphi \le 1.1 \times 10^{-3}$.
With $\varphi = 0.02$: $\rho = 15.5\%$.

Consequence

Exact verifiers carry the safety case. Each verifier publishes (r, φ) from mutation testing (FR-032); consensus is a flag, not a verifier.



Theorem 2: the gate is a derived threshold, not a tuned one

Theorem (Optimal gate, exact)

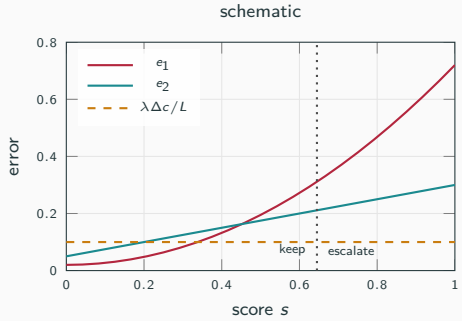
With $e_1(s) = \mathbb{P}[\text{fast wrong} \mid s]$, $e_2(s)$ the error after escalation and $\Delta c(s)$ the extra cost, escalate iff

$$L\big(e_1(s) - e_2(s)\big) > \lambda\, \Delta c(s).$$

Under a budget, the rule is a threshold on $(e_1 - e_2)/\Delta c$ (Neyman–Pearson).

Proposition 1 (price of miscalibration)

Estimation error ϵ on $\hat{e}_1$ costs at most $L\epsilon$ per request: the ECE target 0.05 (NFR-020) is a regret bound.



Same rule as Chow's reject option and the expected value of control; the only tunable is λ, the "speed–care" slider (Prop. 2).

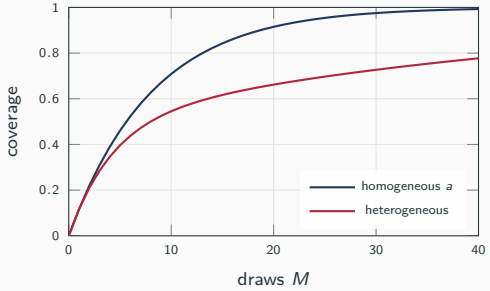
Theorem 3 and Proposition 3: sampling buys coverage; stopping is a Beta rule

Theorem (Coverage, exact)

Draw-verify-stop with acceptance probability a: coverage $1-(1-a)^M$, expected draws $(1-(1-a)^M)/a$, and the error of an accepted answer is $\rho(q)$ for every M.

Proposition (Optimal cap, strong)

With $a \sim \mathrm{Beta}(\alpha, \beta)$, draw again iff $\frac{\alpha}{\alpha+\beta+m} L_a > c_p$ (monotone case), so $M^\star = \lceil \alpha L_a / c_p - \alpha - \beta \rceil$.



Consequence

N and R per domain come from the Phase-0 Beta fit and the cost ratio c_p/L_a (FR-021); heterogeneity of a is measured, not averaged.

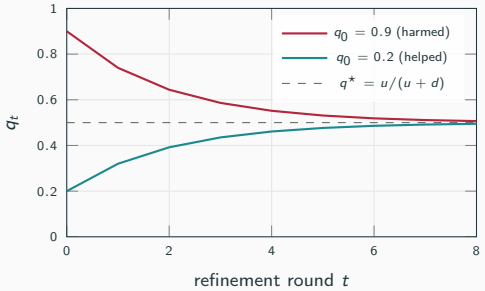
Theorems 4 and 5: the self-monitoring loop must be verifier-gated

Theorem (Unverified refinement, strong)

If a refinement fixes a wrong answer w.p. u and breaks a right one w.p. d, then $q_t = q^\star + (q_0 - q^\star)(1 - u - d)^t$ *with* $q^\star = u/(u+d)$*: iterating harms whenever* $q_0 > q^\star$.

Theorem (Verifier-gated loop, exact)

Accepting only verifier passes gives $\mathbb{P}[\text{accepted wrong}] \le \varphi\,\mathbb{E}[T]$ *and conditional error within* $[\min_t \rho(q_t), \max_t \rho(q_t)]$, *independent of* (u, d).



Matches the empirical finding that LLMs do not self-correct without external feedback. Consequence: no unverified refinement; $R \le 1$ on non-verifiable tasks (FR-022); the prompt can change without a new safety case.

Theorem 6, Propositions 4–8: consensus floor, coupling, audits, sizes, roofline

Plurality voting (exact)

Converges to the modal answer: limiting accuracy $= \mathbb{P}_x[\text{mode correct}]$. Consensus is an assurance level, never `verified`.

Coupling (exact)

Draft-blind slow path: shared-wrong-answer probability is the collision $\sum_y \pi_1 \pi_2$; certifying $< 5\%$ with zero hits needs 59 trials (we use 200).

Audits (exact)

Slow-path accuracy is observed only on escalated tasks; a 5% audit of kept tasks makes the self-model unbiased (SE ≤ 0.019 at $n = 10^4$).

Sample sizes (exact)

DKW: $n \geq \ln(2/\delta)/(2\varepsilon^2)$: 4 600 tasks for ± 2 points; non-inferiority at 1 point needs about 6 200 pairs, at 3 points about 700.

Roofline (exact given the model)

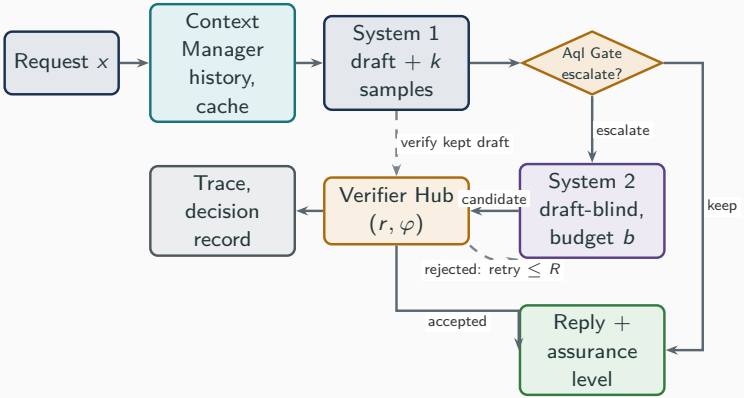
Decode $\leq \beta/(W + K)$: at 38 GB/s a 4B 4-bit model cannot exceed about 15 tok/s; prefill $\leq F/(2P)$. Fixes model sizes, context cap, prompt cache and budget cap.

Distillation (exact)

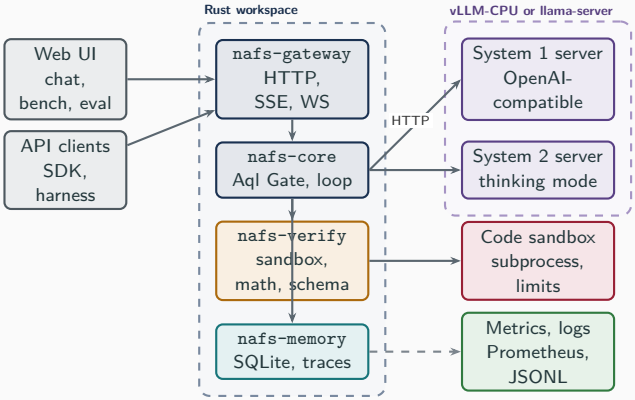
Label noise $\leq \rho$; verifier-measured gains inflate by $1/(r - \varphi)$: promote only on a held-out verifier with confidence bounds.

Architecture

Request lifecycle



System architecture (single node, CPU)



- Rust workspace: `nafs-gateway`, `nafs-core`, `nafs-verify`, `nafs-memory`, `nafs-web`, `nafs-eval`, `nafs-bench`, `nafs-types`
- Backends behind one OpenAI-compatible contract: llama-server (macOS, AVX2) or vLLM-CPU (Linux)
- Every reply: `reasoning_mode`, assurance level, decision record
- GPU scale-out swaps only the model servers

Model profiles (verified on Hugging Face, 2026-10-18)

Profile	System 1	System 2
A (reference)	Qwen3-1.7B, Q4_K_M 1.3 GB, thinking off	Qwen3-4B-Thinking-2507, Q4_K_M 2.3 GB
B (minimal)	Qwen3-4B, `enable_thinking=false`	same weights, thinking on
C (upgrade)	Qwen3.5-2B	Qwen3.5-4B (hybrid attention; gate G0 decides)

Facts that shaped the choice

- Only Qwen3 and Qwen3.5 have open weights $\leq$9B; Qwen3.6/3.8 start at 27B
- Official GGUF exists only for Qwen3; Qwen3-1.7B/4B are on vLLM's validated CPU list
- No official non-autoregressive Qwen exists

Why not Qwen3.5 by default

Hybrid Gated-DeltaNet CPU kernels are newer; vLLM's accelerated path needs AVX-512-BF16 or AMX; no controlled AVX2 benchmark exists. Hypothesis H7 (decode $\geq$ 70% of dense) is measured in Phase 0.

Stack and feasibility

Implementation stack (verified against current documentation)

Serving

- vLLM: x86 CPU backend needs Linux; AVX-512 recommended, AVX2 limited; AWQ/GPTQ/INT8; pin 0.31.0
- llama-server v0.7.0: GGUF, logprobs, reasoning budget, n-gram speculation; `n` capped by slots

Fine-tuning

Unsloth 2026.10.6 trains only with a GPU; CPU support means inference and data prep. Path: rented 16 GB GPU, QLoRA (Qwen3) or BF16 LoRA (Qwen3.5), GGUF export, CPU evaluation.

Rust (crates.io, 2026-10-18)

tokio 1.53, axum 0.8.9, tower-http 0.7, reqwest 0.13, serde, schemars 1.2, utoipa 6.0, sqlx 0.9, tracing, metrics, askama + htmx, figment, criterion, goose.

Artefacts

`openapi.yaml` (two linters), 7 JSON Schemas, vLLM and llama-server configs, Cargo workspace, sample TOML.

Measured on the reference machine (i3-8100, 4 cores, AVX2, 32 GB)

Model (GGUF, llama.cpp 797de63)	Size	Prefill	Decode
Qwen3-0.6B Q8_0	604 MiB	180.6	32.6
Qwen3-1.7B Q4_K_M	1.26 GiB	94.1	15.5
Qwen3-4B Q4_K_M	2.32 GiB	38.2	6.7
Qwen3-4B-Thinking-2507 Q4_K_M	2.32 GiB	38.4	7.0

tokens/s, 4 threads, pp512/tg128, mean of 3 runs; implied bandwidth ≈20 GB/s.

Probe (llama-server, Qwen3-1.7B)

logprobs returned; 4 parallel samples in 3.0 s with 47/48 prompt tokens cached; thinking toggled per request; a 256-token budget closed the trace and an answer followed; SSE streamed.

Consequence

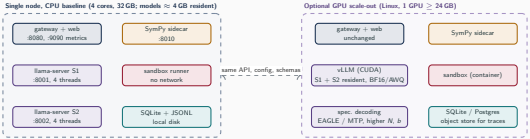
A 1 024-token slow reply takes 2–3 min here: demos and batch evaluation, not interactive.

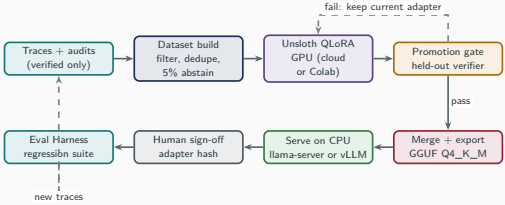
Specification

Specification highlights (99 requirements, RFC 2119)

- **FR-010–015 Gate:** calibrated $\hat{e}_1$, per-domain $\hat{e}_2$, cost ratio; no hand-tuned thresholds in production; failed verification always escalates; decision record per request
- **FR-020–026 Deliberation:** sequential verified sampling; N, R from the Beta fit; draft-blind; budget forcing; $R \leq 1$ on non-verifiable tasks; hard caps
- **FR-030–035 Verification:** exact verifiers at threshold 1.0; published (r, φ); assurance levels; judge hard gate; sandbox limits
- **FR-040–042 Ledger:** 5% audit stream; "unmeasured" labels; CUSUM drift
- **NFR:** wall-clock cap; TTFT ≤ 2 s and 150-token reply ≤ 12 s p95 (fast); slow ≤ 240 s p95 at $b = 1024$; < 8 GB resident; ECE ≤ 0.05; pinned hashes in every trace (targets)
- **IF:** OpenAPI 3.1; SSE events `token/decision/verification/retry/done`; JSON Schemas; backend retry and semaphore policy
- **TR:** verified-only data; Unsloth LoRA r=16; GPU path; promotion gate with confidence bounds and human sign-off
- **SR:** injection delimiting; sandbox isolation; privacy; analogy-only religious wording with scholar review
- Traceability matrix: requirement $\rightarrow$ result $\rightarrow$ component $\rightarrow$ test $\rightarrow$ phase

Deployment and training pipeline

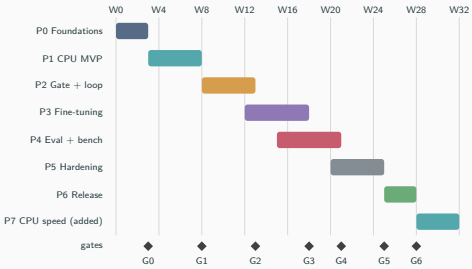




Training runs on a rented GPU; everything else, including the promotion gate and all evaluation, runs on CPU. The GPU scale-out keeps the gateway, verifiers, memory and UI unchanged.

Roadmap

Eight phases, seven gates (estimates)



- **P0** serving, 4 600-task baseline, verifier rates, H4/H7 G0
- **P1** CPU MVP: gateway, both paths, fixed-rule gate, chat UI G1
- **P2** calibrated gate, verified loop, audits; 3-point non-inferiority G2
- **P3** Unsloth cycle under the promotion gate G3
- **P4** evaluation suite, performance console, drift G4
- **P5** hardening, deployment, optional GPU and Profile C G5
- **P6** 1-point non-inferiority, scholar review, publication G6
- **P7** CPU inference performance: latency attribution, engine A/B, early escalation, EAGLE-3 (added)

61 person-weeks, 42 tasks, every requirement scheduled; backlog exported as CSV and Markdown.

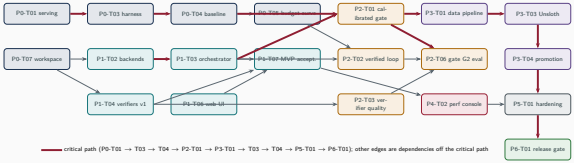
MVP and critical path

MVP (Phase 1)

Profile A on the reference machine; gateway with chat and SSE; fixed-rule gate with forced escalation on failed verification; code and schema verifiers; draft-blind slow path with budget forcing; assurance levels; decision records; chat UI. Nine acceptance tests (MVP-A1..A9).

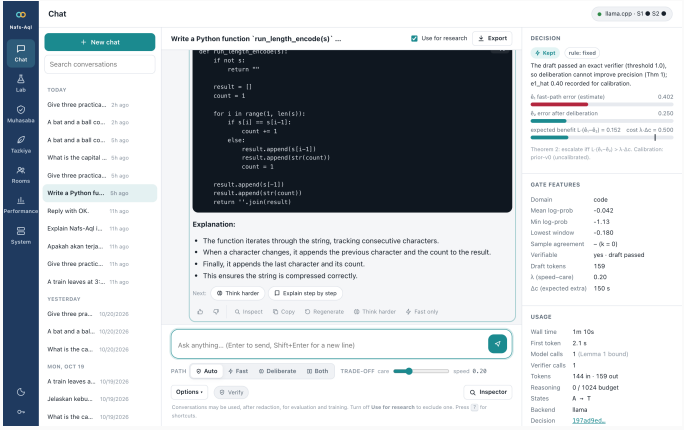
Critical path

serving → baseline → calibrated gate → data pipeline → training → promotion → hardening → release.



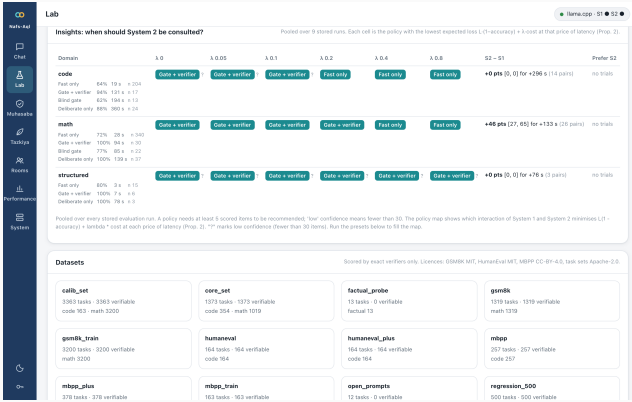
The system as built

The system as built: chat with an inspectable cascade



- Rust workspace of 8 crates; 50 documented endpoints (OpenAPI 1.5); OpenAI-compatible chat
- System 1 Qwen3-1.7B, System 2 Qwen3-4B-Thinking-2507, Q4_K_M on llama-server (vLLM-CPU profile in a VM)
- Each answer shows the strip A→L→M→T/X, its assurance and the gate's reason; the inspector shows the decision, features and a per-stage latency waterfall
- Here: the draft passed the code tests, so the gate kept it (Theorem 1)
- *Both* trials: a person compares System 1 and System 2 answers on the same prompt

The laboratory: when should System 2 be consulted?



- Each combination of the two systems is a *policy*: fast only, deliberate only, gate with verifier, blind gate, draft-conditioned ablation, budget and λ sweeps
- The policy map pools every run: per domain and price of latency λ, the policy with the lowest $L(1 - \mathrm{acc}) + \lambda c$
- Now (6 runs): code: the gate with the verifier for $\lambda \leq 0.1$, System 1 alone for $\lambda \geq 0.2$; math: the gate with the verifier for $\lambda \leq 0.1$, System 1 alone for $\lambda \geq 0.2$; structured: the gate with the verifier at every λ
- Paired bootstrap CIs, exact McNemar, Bonferroni; exports to Markdown and CSV

Measured results (indicative)

Policies (run `7ffd4442`)

System	n	Acc.	95% CI	Esc.	Wall (s)
fast_only	6	67%	[22%, 96%]	0%	13
nafs_aql	6	100%	[54%, 100%]	33%	96
nafs_blind	6	67%	[22%, 96%]	17%	133
slow_conditioned	6	100%	[54%, 100%]	100%	319
slow_only	6	100%	[54%, 100%]	100%	264

Gate with verifier = deliberate-only accuracy at 0.36 of the cost; the blind gate routed 50% correctly (samples agreed with wrong drafts).

Verifiers (r true accept, φ false accept)

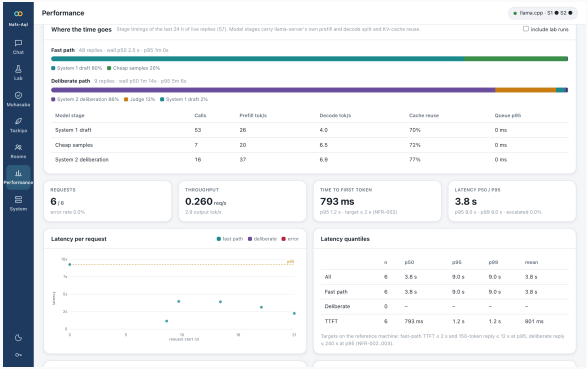
Verifier	Correct	r	Mutants	φ	φ upper
code_tests	364	0.995	754	0.109	0.133
json_schema	24	1.000	53	0.000	0.067
judge	23	1.000	22	0.045	0.228
math_numeric	2000	1.000	2800	0.000	0.001

Profile C (System 1 candidates, 30 tasks)

System 1	All	Math	Code	JSON	Wall (s)	tok/s
Qwen3-1.7B (Profile A)	67%	56%	75%	83%	24.0	4.3
Qwen3.5-2B (Profile C)	83%	81%	75%	100%	88.9	1.9

Qwen3.5-2B: $+17$ points, 95% CI $[+7, +30]$, $n = 30$, but $3.7\times$ slower $\Rightarrow$ Profile A stays.

Phase 7: where the time goes on a CPU



- Every reply records its stages; model stages carry llama-server's prefill, decode and KV-cache reuse
- Fast path ≈100% System 1 draft; deliberate path ≈97% System 2 reasoning; the prompt cache serves most prompt tokens ⇒ **decode speed is the bottleneck**
- Shared host: 2.6–4.3 tok/s in service vs 22.9 idle

Model stage	Calls	Prefill tok/s	Decode tok/s	Cache reuse
System 1 draft	485	53	7.0	60%
cheap samples	32	13	4.2	80%
System 2 deliberation	45	16	3.4	95%

Adopted after interleaved A/B

Skip samples when a verifier decides (13.4→7.9 s); samples only when the draft is uncertain (factual 12.6→4.3 s); prompt-cache and verifier warm-up (TTFT 9.8→1 s); early escalation; 4 System 1 slots

Early escalation, concurrency, and the next engine levers

Early escalation (FR-080)

Stop an open-ended draft once its running mean log-prob is below the fixed threshold minus a margin; System 2 takes over. 40 prompts: N = 32, margin 0.04 agrees 100%, catches 3/10 escalations, saves ≈116 tokens each.

N	Margin	Agreement	Caught	False	Saved tok/esc.
32	0.00	80%	10/10	8	82
32	0.02	95%	5/10	2	92
32	0.04	100%	3/10	0	116
32	0.15	100%	0/10	0	0
48	0.00	90%	9/10	4	77
48	0.02	98%	4/10	1	117
48	0.04	100%	2/10	0	154
48	0.15	100%	0/10	0	0

Concurrency (NFR-007)

Latest protocol run 6c462074: ratio 1.25. Gate G4 uses the median over eligible runs: median ratio 2.15 over 3 run(s) (1.25, 2.15, 2.48); fallback: document the single-session profile.

Adaptive reasoning exit (FR-082, new)

System 2 reasons in chunks; at reflection points its answer is probed and the exact verifier decides whether to stop. Paired Lab run: with the exit: accuracy 100%, mean wall 123 s; without: 100%, 220 s ($n = 11$ GSM8K items; paired Δ accuracy +0 points, 95% CI [+0, +0]).

Measured, no gain, not adopted

0.6B draft and n-gram speculation; flash attention, q8_0 KV; EAGLE-3 heads (1.04× / 0.99×); System 1 as judge (φ 0.27 vs 0.045).

Backends and weight formats: measured, not assumed

vLLM CPU backend in Docker (new)

`carstenz/vllm-smoke` image (vLLM 0.30, x86 CPU build), one model in two modes on :8101; the gateway switches backend sets at run time.

- AVX2-only VM: BF16 loads but never returns a token (no oneDNN kernels) $\Rightarrow$ FP32
- vLLM's three processes cost $\approx$3 GB beyond the weights: 1.7B needs an 11–12 GiB VM, 0.6B about 7
- Gateway check (every orchestrator path): no result yet: on the day of the work the host had under 1 GB free and the Docker VM did not start at the 7.5 GiB the stand-in needs

Weight formats (llama-bench, Qwen3-1.7B)

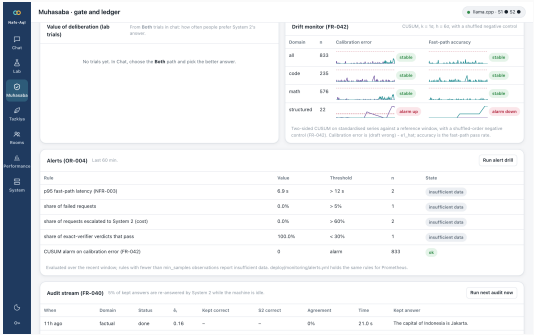
Weights (Qwen3-1.7B)	Prefill tok/s	Decode tok/s
Qwen3-1.7B-Q4_K_M	81	15.3
Qwen3-1.7B-Q4_0 (own)	44	12.2
Qwen3-1.7B-IQ4_NL (own)	70	16.0
Qwen3-1.7B-Q4_K_M (own)	72	12.8

The repack formats buy nothing over Q4_K_M on AVX2; the format stays.

A smaller judge?

System 2 judge: $r = 1.00$, $\varphi = 0.045$ (upper 0.23); System 1 judge: $r = 0.78$, $\varphi = 0.27$ (upper 0.50); 23 correct and 22 seeded wrong answers. Accepting a quarter of wrong answers is not a verifier (Thm. 1): the judge stays on System 2.

Muhasaba: drift, alerts, security and the gates



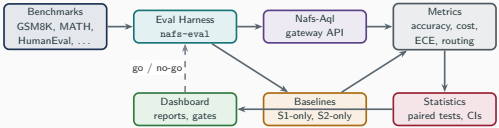
CUSUM drift per domain with a shuffled negative control; five alert rules over live traffic (lab traffic tagged). 26/26 security tests pass; 0 open high-severity findings; deployment checks: 11 passed, 0 failed, 1 skipped (tool or host not available).

Gate	Status	Evidence
G0	pending	H7: pass; P0-T04: pending
G1	fail	MVP: fail; NFR-002/004: fail
G2	pending	ECE: pending; H1: pending; ER-002: pending; ER-004: pending
G3	pending	TR-007: pending; OR-005: pending
G4	fail	P4: pass; ER-007: pass; NFR-007: fail
G5	pass	SR-006/SR-002: pass; OR-003: pass; MR-005: pass
G6	pending	ER-002: pending; ER-008: pass; SR-005: pending

Evaluation

Evaluation plan and statistically sized gates

- **Metrics:** accuracy, selective accuracy and coverage, escalation rate, routing accuracy vs the oracle rule, cost ratio and cost–accuracy curve, ECE (5 bins, $n \geq 2\,900$), verifier (r, φ) with intervals, adoption rate, robustness, latency quantiles
- **Benchmarks:** GSM8K, MATH, HumanEval, MBPP, schema tasks, MMLU, GPQA, TruthfulQA, SimpleQA, BBH, LiveCodeBench, MT-Bench style; RouteLLM data for calibration
- **Protocol:** paired, seeded, bootstrap CIs, non-inferiority (3 points at G2, 1 point at G6), Bonferroni across domains, CUSUM drift with negative control



Hypotheses H1–H7

Gate accuracy; cost ratio; Beta fit; concave budget curve; coupling; distillation gain; hybrid CPU speed. Each has an experiment, a phase and an engineering effect if false.

Risks, assumptions, and the first three actions

Top risks

- Slow path not interactive on CPU (mitigate: smaller budget, Profile B, GPU profile)
- Gate calibration may fail on uninformative features (budget mode, richer features)
- Verifier false accepts (hidden tests, mutation testing)
- Correlated errors across shared weights (exact verifiers carry the case)
- Version drift of models and runtimes (pins, hashes, regression suite)

Key assumptions

CPU-only means the i3-8100 class; Unsloth runs on a rented GPU; vLLM on Linux, llama-server elsewhere, same API; the draft's missing companion documents are inferred from the deck.

First three actions

1. Run the Phase-0 baseline ($\geq 4\,600$ tasks) and fit e_1, e_2, Beta, budget curves *(harness done; only scaled runs on this CPU)*
2. Mutation-test the code, math and schema verifiers; publish (r, φ) *(done)*
3. llama-bench Qwen3.5-2B/4B vs Qwen3-1.7B/4B on the target CPU *(done: H7 passes; Profile C not adopted)*

The road ahead (beyond Phase 7)

Phase 8: evidence at scale

Faster or idle host (or nightly slices): the 4 600-task baseline, 700 paired items for G2, 2 900 labelled drafts to promote the calibrated gate and budget mode, 6 200 pairs for G6.

Phase 9: Tazkiya cycle

5 000 verified traces, the first Unsloth LoRA on a rented GPU, promotion under TR-007, rollback drill; re-fit the EAGLE-3 head to the distilled System 1.

Phase 10: engine

EAGLE-3 / MTP speculation; repacked 4-bit formats; adaptive early exit from System 2 reasoning; a learned router from the policy map and Both-trial preferences.

Phase 11: product and review

Scholar review of the analogy text (SR-005); GPU profile for interactive deliberation; multilingual UI (Indonesian, Arabic); open-domain evidence checks from retrieval.

Nafs-Aql: deliberate when it pays, accept only on evidence, keep honest accounts.